Top 50 React Tips Every Developer Should Know

Whether you're already familiar with React or building your way up from the basics, there's always room to improve your workflow and code efficiency.

Here are 50 React tips to help you write better React code.

1. Use `useRef` hook to Persist Value Across Renders

Instead of using the `useState` hook to store value, use the useRef hook, If you don't want to re-render the component on setting value.

Use Case: Storing timeouts, intervals, DOM references, or any value that doesn't need to trigger re-renders, to avoid closure issues.

```
const interval = useRef(-1);

useEffect(() => {
  interval.current = setInterval(() => {
    console.log('Timer executed');
  }, 1000);

  return () => clearInterval(interval.current);
}, []);
```

---

2. Avoid Unintended Rendering with Logical AND (&&) in JSX

Don't make the mistake of writing JSX like this:

```
{ users.length && <User /> }
```

In the above code, If the `users.length` is zero then we will end up displaying `0` on the screen.

Because the logical `&&` operator(also known as short circuit operator) instantly returns that value if it evaluates to a falsy value.

So the above code will be equivalent to the below code if `users.length` is `0`.

```
{ 0 && <User /> }
```

which evaluates to `{0}`

Instead, you can use either of the following ways to correctly write the JSX:

```
{ users.length > 0 && <User /> }
{ !!users.length && <User /> }
{ Boolean(users.length) && <User /> }
{ users.length ? <User /> : null }
```

I usually use the first approach of checking `users.length > 0`.

However, you can use any of the above ways.

The `!!` operator converts any value into its boolean `true` or `false`.

So the above 2nd and 3rd conditions are equivalent.

The ternary operator `?` in the fourth case above will check for the truthy or falsy value of the left side of `?` and as `0` is falsy value, so `null` will be returned.

And as you might know, React will not display `null`, `undefined` and boolean values on the UI when used in JSX expression like this:

```
{ null }
{ undefined }
{ true }
{ false }
```

If you want the boolean values to display on screen, you need to convert it into a string like this:

```
const isLoggedIn = true;

{ "" + isLoggedIn }

OR

{ `${isLoggedIn}` }

OR

{ isLoggedIn.toString() }
```

Check out part two of this series here.

---

3. Avoid declaring/nesting Components Inside Other Components

While it may seem convenient at first, this practice can cause major performance issues in your application.

Every time the parent component is re-rendered because of a state update, the nested component is re-created, which means the previous state and data declared in that nested component are lost causing major performance issues.

```
// Wrong Way:
const UsersList = () => {

  ❌ Declaring User component inside UsersList component
  const User = ({ user }) => {
      // some code
    return <div>{/* User Component JSX */}</div>
  };

  return <div>{/* UsersList JSX */}</div>
};

// Right Way:
✅ Declaring User component outside UsersList component
const User = ({ user }) => {
  // some code
  return <div>{/* User Component JSX */}</div>
};

const UsersList = ({ users }) => {
  // some code
  return <div>{/* UsersList component JSX */}</div>
};
```

---

4. Use Lazy Initialization of State with `useState`

For expensive initial state computations, you can delay the evaluation until the component is rendered by passing a function to `useState`.

This will ensure the function only runs once during the initial render of the component. Check out the application that uses `useLocalStorage` hook.

```
const useLocalStorage = (key, initialValue) => {

  const [value, setValue] = useState(() => {
    // lazy initialization of state
    try {
      const localValue = window.localStorage.getItem(key);
      return localValue ? JSON.parse(localValue) : initialValue;
    } catch (error) {
      return initialValue;
    }
  });

  useEffect(() => {
    window.localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
};
```

---

5. Memoize Expensive Functions with `useMemo`

For expensive computations that don't need to run on every render, use `useMemo` hook to memoize the result and improve performance.

The first code shown below causes a performance issue because, on every re-render of the component, the `users` array is filtered and sorted again and again.

And If the `users` array contains a lot of data, then it will slow down the application.

But If we use the `useMemo` hook, the `users` array will be filtered and sorted only when the `users` dependency is changed.

```jsx
// Instead of writing code like this:
function App({ users }) {
  return (
    <ul>
      {users
        .map((user) => ({
          id: user.id,
          name: user.name,
          age: user.age,
        }))
        .sort((a, b) => a.age - b.age)
        .map(({ id, name, age }) => {
          return (
            <li key={id}>
              {name} - {age}
            </li>
          );
        })}
    </ul>
  );
}

// write it like this:
function App({ users }) {

  const sortedUsers = useMemo(() => {
    return users
      .map((user) => ({
        id: user.id,
        name: user.name,
        age: user.age,
      }))
      .sort((a, b) => a.age - b.age);
  }, [users]);

  return (
    <ul>
      {sortedUsers.map(({ id, name, age }) => {
        return (
          <li key={id}>
            {name} - {age}
          </li>
        );
      })}
    </ul>
  );
}
```

6. Use Custom Hooks for Reusable Logic

Custom hooks are a powerful way to extract reusable logic from your components. If you find yourself copying and pasting the same code in multiple components, create a custom hook.

```
function useFetch(url) { // custom hook to get data from API by passing url
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch(url)
      .then(response => response.json())
      .then(data => setData(data));
  }, [url]);

  return data;
}
```

7. Use Fragments to Avoid Extra DOM Nodes

Wrapping components with unnecessary `<div>` elements can lead to a bloated DOM. Use fragments instead.

This keeps your DOM clean and optimized for performance.

Note that, every time you add a new `<div>`, React has to create that element, using `React.createElement` so avoid creating unnecessary `<div>`.

```
// Instead of writing like this:
return (
  <div>
    <Child1 />
    <Child2 />
  </div>
);

// write like this:
return (
  <>
    <Child1 />
    <Child2 />
  </>
);

// or like this:
return (
  <React.Fragment>
    <Child1 />
    <Child2 />
  </React.Fragment>
);
```

---

8. Leverage Compound Components for Flexible UIs

Compound components let you create flexible UIs by allowing components to communicate implicitly.

It’s a great way of building reusable and highly customizable components.

```
function Tab({ children }) {
  return <div>{children}</div>;
}

Tab.Item = function TabItem({ label, children }) {
  return (
    <div>
      <h3>{label}</h3>
      <div>{children}</div>
    </div>
  );
};

// Usage
<Tab>
  <Tab.Item label="Tab 1">Content 1</Tab.Item>
  <Tab.Item label="Tab 2">Content 2</Tab.Item>
</Tab>
```

---

9. Always Add `key` Prop While Rendering Using map Method

Always add a unique `key` prop to items rendered in a list to help React optimize rendering.

> *The value of `key` should remain the same and should not change on component re-render.*

Always have a unique ID for each element of the array which can be used as a key.

Avoid using an array `index` as a value for `key`.

Using an array index as `key` prop can lead to unexpected UI behavior when array elements are added, removed, or reordered.

```
// ✅ Use unique id as a key
{items.map(item => (
  <Item key={item.id} item={item} />
))}

// ❌ Don't use index as the key
{items.map((item, index) => (
  <Item key={index} item={item} />
))}
```

---

10. Use Error Boundary To Avoid Breaking Your Application On Production

React Error Boundaries provide a powerful mechanism to catch and gracefully handle errors that occur during the rendering of your components.

They help prevent your entire application from crashing by isolating errors to specific components, allowing the rest of your app to continue functioning.

```
<ErrorBoundary
   FallbackComponent={ErrorPage}
   onReset={() => (location.href = '/')}
 >
   <App />
</ErrorBoundary>
```

---

11. Avoid Prop Drilling by Using Context

Instead of passing props through many nested components, use React Context API for more manageable data sharing.

Best Use Case: Sharing global state like theme, user data, or settings.

```jsx
export const AuthContext = React.createContext();

export const AuthContextProvider = ({ children }) => {
  const [user, setUser] = useState(null);

  const updateUser = (user) => {
    setUser(user);
  };

  return (
    <AuthContext.Provider value={{ user, updateUser }}>
      {children}
    </AuthContext.Provider>
  );
};

// Usage
<AuthContextProvider>
  <App />
</AuthContextProvider>
```

12. Use the Dependency Array In `useEffect` Correctly

Always include every outside variable referenced inside `useEffect` in the dependency array `[]` to avoid bugs caused by stale closures.

```jsx
useEffect(() => {
  const fetchData = (id) => {
    // some code to fetch data
  }
  id && fetchData(id);
}, [id]);
```

13. Use React Suspense for Code Splitting

React's Suspense lets you split your code into smaller chunks, improving performance by loading components only when needed.

```
import { Suspense, lazy } from 'react';
import { BrowserRouter, Route, Routes } from 'react-router-dom';

const Dashboard = lazy(() => import('./pages/dashboard'));
const Profile = lazy(() => import('./pages/profile'));

const App = () => {
  return (
    <BrowserRouter>
      <Suspense fallback={<p>Loading ... </p>}>
        <Routes>
          <Route path='/dashboard' element={<Dashboard />} />
          <Route path='/profile' element={<Profile />} />
        </Routes>
      </Suspense>
    </BrowserRouter>
  );
};
```

In the above code, `Profile` and `Dashboard` component code will not be loaded initially, instead, it will be loaded only when `/profile` route or `/dashboard` route is accessed respectively.

---

14. Use `StrictMode` for Highlighting Potential Issues

Enable React's `StrictMode` in development to identify potential problems in your app like unsafe lifecycles, deprecated methods, or unexpected side effects.

Enabling `StrictMode` shows a warning in the browser console which helps to avoid future bugs.

```
<React.StrictMode>
  <App />
</React.StrictMode>
```

15. Manage State Locally Before Lifting It Up

Avoid lifting state to parent components unnecessarily.

If a piece of state is only used by a single component or closely related components, keep it local to avoid over-complicating the state management.

Because whenever the state in the parent component changes, all the direct as well as indirect child components get re-rendered.

```
function ChildComponent() {
  const [value, setValue] = useState(0);

  return <div>{value}</div>;
}
```

16. Prefer Functional Components over Class Components

Always use functional components with hooks instead of class components.

Functional components offer more concise syntax, and better readability, and make it easier to manage side effects.

They're also a bit faster than class components because they don't have inherited life cycle methods, and extra code of parent inherited classes

```
function MyComponent() {
  const [state, setState] = useState(0);

  return <div>{state}</div>;
}
```

17. Use IntersectionObserver for Lazy Loading Elements

Use Intersection Observer API to load images, videos, or other elements only when they're visible on the screen.

This improves performance, especially in long lists or media-heavy apps.

```
useEffect(() => {
  const observer = new IntersectionObserver((entries) => {
    entries.forEach(entry => {
      if (entry.isIntersecting) {
        // Load image or other data
      }
    });
  });
  observer.observe(elementRef.current);
}, []);
```

18. Split Large Components into Smaller Ones

Break down large components into smaller, reusable components for improved readability and maintainability.

Aim for components that follow the "single responsibility" principle.

Smaller components are easier to test, maintain, and understand.

```
const Header = () => <header>{/* Header Component Code */}</header>;

const Footer = () => <footer>{/* Footer Component Code */}</footer>;
```

19. Avoid Unnecessary useEffect and setState Calls

Keep in mind that the code written in the `useEffect` hook will always execute after the render cycle, and calling `setState` inside `useEffect` triggers a re-render of your component again.

Additionally, if the parent component re-renders, all child components will also re-render. To avoid unnecessary re-renders, always aim to minimize extra `useEffect` hooks and avoid redundant `setState` calls when possible.

```
// Instead of writing code like this:
const [username, setUsername] = useState("");
const [age, setAge] = useState("");
const [displayText, setDisplayText] = useState("");

useEffect(() => {
 setDisplayText(`Hello ${username}, your year of birth is ${new Date().getFullYear
() - age}` );
}, [username, age]);

// write it like this:

const [username, setUsername] = useState("");
const [age, setAge] = useState("");

// create local variable in component
const displayText = `Hello ${username},
your year of birth is ${new Date().getFullYear() - age}`;
```

With the above code, the `displayText` will always be updated when the `username` or `age` is changed and it avoids extra re-renders of your component.

20. Avoid Writing Extra Functions In Components If Possible

Every function declared inside a functional component gets recreated on every re-render of the component.

So, If you have declared separate functions for showing/hiding a modal that manipulates the state like this:

```
const [isOpen, setIsOpen] = React.useState(false);

function openModal() {
  setIsOpen(true);
}

function closeModal() {
  setIsOpen(false);
}
```

Then, you can simplify this logic by creating a single `toggleModal` function that handles both opening and closing the modal.

```
const [isOpen, setIsOpen] = React.useState(false);

function toggleModal() {
  setIsOpen((open) => !open);
}
```

So, by just calling the `toggleModal` function, the modal will be closed, If it's open and vice-versa.

---

21. Always Assign Initial Value For useState Hook

Whenever you're declaring a state in a functional component, always assign an initial value like this:

```
// set initial value of an empty array
const [pageNumbers, setPageNumbers] = useState([]);
```

Never declare any state without any default value like this:

```
const [pageNumbers, setPageNumbers] = useState();
```

It makes it difficult to understand what value to assign when you have a lot of states and you're trying to set the state using the `setPageNumbers` function later in the code.

If you're working in a team, one developer might assign an array in place using `setPageNumbers([1, 2, 3, 4])` and another might assign a number like `setPageNumbers(20)`.

If you're not using TypeScript, it might create a hard-to-debug issue which is caused just because of not specifying an initial value.

So always specify the initial value while declaring a state.

---

22. Understanding State Updates and Immediate Access in React

In React, it's important to remember that the state does not get updated immediately after setting it. This means that if you try to access the state value immediately after setting it, you might not get the updated value.

For instance, consider the following code where we attempt to log the updated state right immediately after calling `setIsOnline`.

```
const [isOnline, setIsOnline] = React.useState(false);

function toggleOnlineStatus() {
  setIsOnline(!isOnline);
  console.log(isOnline); // Logs the old value, not the updated one
}
```

The same happens with the below code also:

```
const [isOnline, setIsOnline] = React.useState(false);

function doSomething() {
  if(isOnline){ // isOnline value is still old and not updated yet
    // do something
  }
}

function toggleOnlineStatus() {
  setIsOnline(!isOnline);
  doSomething();
}
```

React updates state during the next render, after it has done executing all the code from the current function.

So in the above `doSomething` function, `isOnline` will still have the old value and not the updated value.

---

23. Avoid Using Context API In Every Case

Passing props down to components one or two levels deep is not an issue. Props are designed to be passed to components, so there's no need to use the Context API just to avoid passing those props.

Many React developers use the Context API simply to avoid passing props, but that's not the intended use case. The Context API should be used only when you want to avoid **prop drilling**, which refers to passing props to deeply nested components.

---

24. Always Use `null` As The Value To Store Objects In State

Whenever you want to store an object in a state, use `null` as the initial value like this:

```
const [user, setUser] = useState(null);
```

instead of an empty object like this:

```
const [user, setUser] = useState({});
```

So if you want to check if a `user` exists or if there are any properties present inside it, you can just use a simple if condition like this:

```
if (user) {
  // do something
}
```

or use JSX expression like this:

```
{ user && <p>User found</p>}
```

However, If you use an empty object as the initial value, then to find out If `user` exists or not, you need to check like this:

```
if(Object.keys(user).length === 0) {
  // do something
}
```

or use JSX expression like this:

```
{ Object.keys(user).length === 0 && <p>User found</p>}
```

---

25. Always Create `ContextProvider` Component Instead Of Directly Using `Context.Provider`

Whenever using React Context API, don't pass `value` prop directly to `Provider` as shown below:

```
const App = () => {
 ....
 return (
   <AuthContext.Provider value={{ user, updateUser }}>
      <ComponentA />
      <ComponentB />
   </AuthContext.Provider>
 );
}
```

In the above code, whenever the App component re-renders because of state update, the `value` prop passed to the `AuthContext.Provider` will change,

and because of that, all the direct as well as indirect components in between the opening and closing tag of `AuthContext.Provider` will get re-rendered unnecessarily.

So, always create a separate component as shown below:

```
export const AuthContextProvider = ({ children }) => {
  const [user, setUser] = useState(null);

  const updateUser = (user) => {
    setUser(user);
  };

  return (
    <AuthContext.Provider value={{ user, updateUser }}>
      {children}
    </AuthContext.Provider>
  );
};
```

and use it as shown below:

```
<AuthContextProvider>
  <ComponentA />
  <ComponentB />
</AuthContextProvider>
```

In React, `children` is a special prop, so even if the parent component re-renders, any component passed as a `children` prop will not get re-rendered.

So when you write code like this:

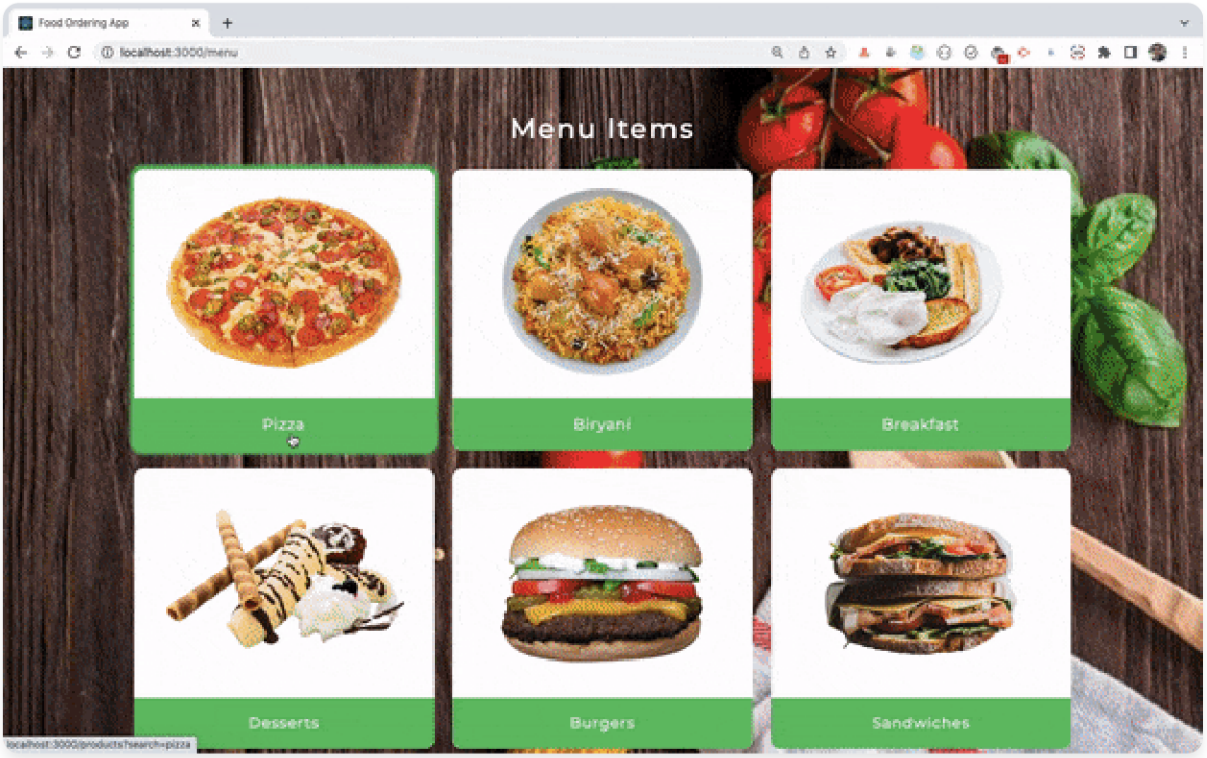
```
<AuthContextProvider>
  <ComponentA />
  <ComponentB />
</AuthContextProvider>
```

then the components wrapped in between the opening and closing tag of `AuthContextProvider` will only get re-rendered if they're using the `useContext` hook, otherwise, they will not get re-rendered even if the parent component re-renders.

So to avoid unnecessary re-rendering of all child components, always create separate component to wrap the `children` prop.

---

26. Use LocatorJS Browser Extension To Quickly Navigate Between Source Code



Use LocatorJS browser extension to quickly navigate to the source code in your favorite IDE (VS Code) by just clicking on any of the UI elements displayed in the browser.

This extension is really useful when you have a large application and you want to find which file contains the code of the displayed UI.

It's also useful when the code is written by someone else and you want to debug the code.

Use **Option/Alt + Click** to navigate to the source code.

This extension becomes a lifesaver when you're working with Material UI library code written by others and you want to find out the exact code for any part of the UI.

---

27. Never Hardcode API URL In The Application

Whenever you're making an API call, never hardcode the API URL, instead, create `src/utils/constants.js` or a similar file and add the `API URL` inside it like this:

```
export const BASE_API_URL = "https://jsonplaceholder.typicode.com";
```

and wherever you want to make an API call, just use that constant using template literal syntax like this:

```
// posts.jsx

import { BASE_API_URL } from '../utils/constants.js';

const {data} = await axios.get(`${BASE_API_URL}/posts`);

// users.jsx

import { BASE_API_URL } from '../utils/constants.js';

const {data} = await axios.get(`${BASE_API_URL}/users`);
```

And with this change, If later you want to change the `BASE_API_URL` value, then you don't need to change it in multiple files.

You just need to update it in the `constants.js` file, it will get reflected everywhere.

PS: You can even store the `BASE_API_URL` value in the `.env` file instead of storing it in `constants.js` file so you can easily change it depending on the development or production environment.

---

28. Use Barrel Exports To Organize React Components

When you're working on a large React project, you might have multiple folders containing different components.

In such cases, If you're using different components in a particular file, your file will contain a lot of import statements like this:

```
import ConfirmModal from './components/ConfirmModal/ConfirmModal';
import DatePicker from './components/DatePicker/DatePicker';
import Tooltip from './components/Tooltip/Tooltip';
import Button from './components/Button/Button';
import Avatar from './components/Avatar/Avatar';
```

which does not look good because as the number of components increases, the number of import statements will also increase.

To fix this issue, you can create an `index.js` file in the parent folder `(components)` folder and export all the components as a named export from that file like this:

```
export { default as ConfirmModal } from './ConfirmModal/ConfirmModal';
export { default as DatePicker } from './DatePicker/DatePicker';
export { default as Tooltip } from './Tooltip/Tooltip';
export { default as Button } from './Button/Button';
export { default as Avatar } from './Avatar/Avatar';
```

This needs to be done only once. Now, If in any of the files you want to access any component, you can easily import it using named import in a single line like this:

```
import {ConfirmModal, DatePicker, Tooltip, Button, Avatar} from './components';
```

which is the same as

```
import {ConfirmModal, DatePicker, Tooltip, Button, Avatar} from './components/inde
x';
```

This is standard practice when working on large industry/company projects.

This pattern is known as the barrel pattern which is a file organization pattern that allows us to export all modules in a directory through a single file.

Here's a CodeSandbox Demo to see it in action.

29. Storing API Key In .env file In React Application Do Not Make It Private

If you're using a `.env` file to store private information like `API_KEY` in your React app, it does not actually make it private.

Your React application runs on the client side/browser so anyone can still see it If that `API_KEY` is used as a part of the `API URL` from the network tab.

The better way to make it private is to store it on the backend side.

Note that, even though using the `.env` file within React doesn't guarantee absolute privacy, it's advisable to follow this practice for an added layer of security.

So nobody will be able to find out the `API_KEY` directly in your application source code.

Also, make sure not to push `.env` file to GitHub for security reasons. You can achieve that by adding it to the `.gitignore` file.

---

30. Passing a Different key To Component Causes Component Re-Creation

```
<BookForm
  key={book.id}
  book={book}
  handleSubmit={handleSubmit}
/>
```

Whenever we pass a different key to the component, the component will be re-created again and all its data and state will be reset.

This is sometimes useful If you don't want to retain the previous information / reset the component state, after some action.

For example, on a single page, you're opening different modals to display different information, then you can pass a unique key for each modal, so every time you open the modal, you will not have previous information retained and you will be able to display the correct respective modal data.

So that's one of the reasons you need a unique key that will not change during re-render while using the array map method.

Because whenever the key changes, React re-creates that element and all its child elements including components you have nested inside that parent element, causing major performance issues.

---

31. Organize Components by Creating Separate Folders for Each One

Whenever you're creating any component, don't put that component directly inside a components folder.

Instead, create separate folders for individual components inside the components folder like this:

```
--- components
 --- header
   --- Header.jsx
   --- Header.css
   --- Header.test.js
 --- footer
   --- Footer.jsx
   --- Footer.css
   --- Footer.test.js
 --- sidebar
   --- Sidebar.jsx
   --- Sidebar.css
   --- Sidebar.test.js
```

As you can see above, we have created a `header`, `footer`, and `sidebar` folders inside the `components` folder, and inside the `header` folder, all the files related to the `header` are kept.

The same is the case with `footer` and `sidebar` folders.

---

32. Don't Use Redux For Every React Application

Don't use Redux initially itself while building a React app.

For smaller applications using React Context API or useReducer hook is enough to manage the React application.

Following are some of the use cases when you can choose to select Redux.

To Manage Global State: When you have a global application state like authentication, profile info, or data that needs to be shared across multiple components

To Share Data Between Unrelated Components: When your application has multiple pages using routing, and so you can't lift state up the parent component

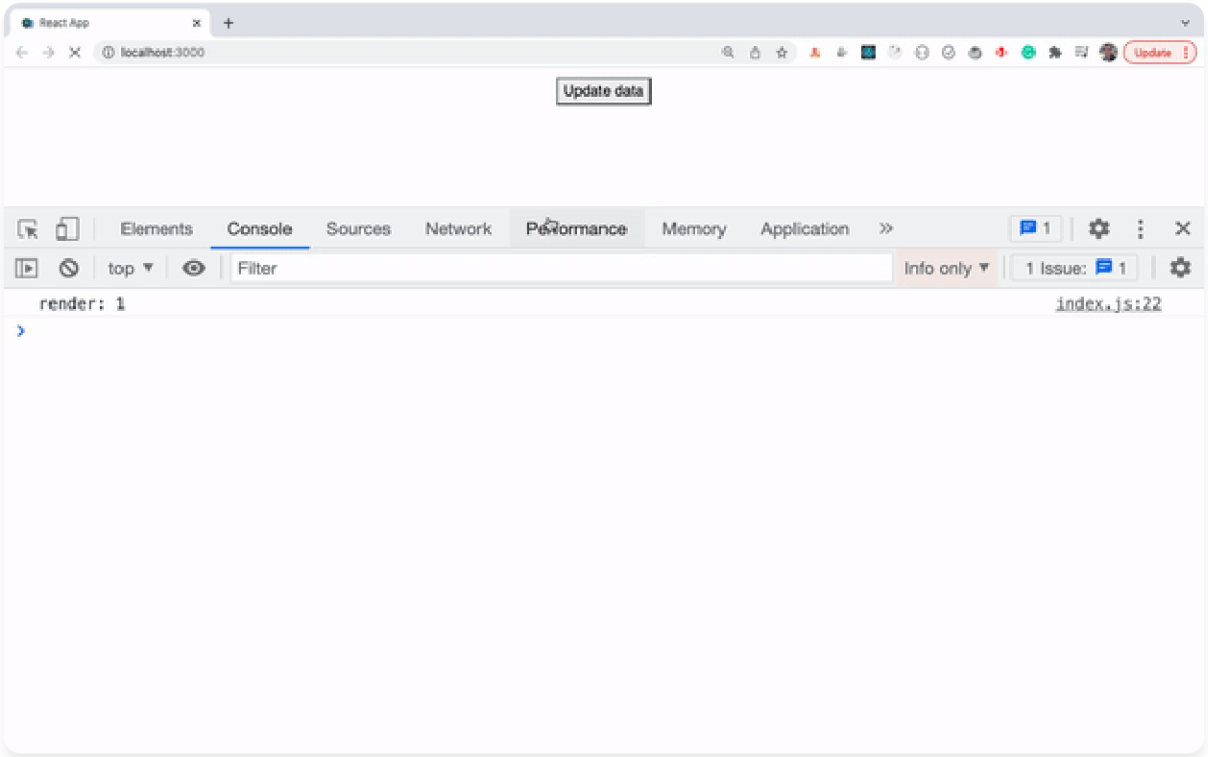
Consistent State Updates: Redux enforces a single source of truth for your application's state, making it easier to update and maintain your application's state in a consistent manner.

Scalability: Redux can help your application scale by providing a clear pattern for managing state as your application grows in size and complexity.

> *You can follow this course to learn Redux + Redux Toolkit from scratch.*

---

33. Use console.count Method To Find Out Number Of Re-renders Of Component



Sometimes we want to know, how many times a particular line of code is executed.

Maybe we want to know how many times a particular function is getting executed.

In that case, we can use a `console.count` method by passing a unique string to it as an argument.

For example, If you have a React code and you want to find out how many times the component is getting re-rendered then instead of adding `console.log` and manually counting how many times it's printed in the console, you can just add `console.count('render')` in the component

And you will see the render message along with the count of how many times it's executed.

---

34. Avoid Declaring A New State For Storing Everything Inside A Component

Before declaring a new state in a component, always think twice If you really need to store that information in the state.

Because once you declare a state variable, you need to call `setState` to update that state value, and whenever the state changes, React will re-render the component holding that state, as well as all of its direct as well as indirect child components.

So by just introducing a single state, you add an extra re-render in your application, and If you have some heavy child components doing a lot of filtering, sorting, or data manipulation, then updating the UI with new changes will take more time.

If you just need to track some value that you're not passing to any child component or not using while rendering then use the useRef hook to store that information.

Because updating the ref value will not cause the component to re-render.

---

35. Don't Declare Every Function Inside A Component

When declaring a function inside a component, If that function doesn't depend on the component's state or props, then declare it outside the component.

Because on every re-render of the component, all the functions in the functional component are re-created again, so If you have a lot of functions in a component, it might slow down the application.

Take a look at the below code:

```
const getShortDescription = (description) => {
  return description.slice(0, 20);
}
```

The above function does not have any dependency on state or prop value.

We're just passing a value to get shortened text. So there is no need to declare it inside the component.

Also, If you need the same short description functionality in different components, then instead of repeating the code in those components, move the function to another file like `utils/functions.js` and export it from there and use it in the required components.

This makes your components easy to understand.

36. Upgrade Your React Apps to At Least Version 18

With React version 18, you get better application performance as compared to a version less than 18.

With version 18, React batches together multiple state updates into a single re-render cycle. So If you have multiple set state calls in a function like this:

```
const handleUpdate = (user) => {
  setCount(count => count + 1);
  setIsOpen(open => !open);
  setUser(user);
}
```

Then just because there are three state update calls, React will not re-render the component three times, instead, it will be rendered only once.

In React version less than 18, only state updates inside event handler functions are batched together but state updates inside `promises`, `setTimeout` function, native event handlers, or any other event handlers are not getting batched.

If you're using a React version less than 18, then for the above code, as there are three state updates, the component will be rendered three times which is performance-in-efficient.

With version 18, state updates inside of `timeouts`, `promises`, native event handlers or any other event will batch together so for the above code rendering happens only once.

---

37. Use Correct Place To Initialize QueryClient In Tanstack Query/React Query

When using Tanstack Query / React Query, never create `QueryClient` instance inside the component.

The `QueryClient` holds the Query Cache, so if you create a new client inside a component, using the below code:

```
const queryClient = new QueryClient();
```

then on every re-render of the component, you will get a new copy of the query cache. so your query data will not be cached and caching functionality will not work as expected.

So always create `QueryClient` instance outside the component instead of inside the component.

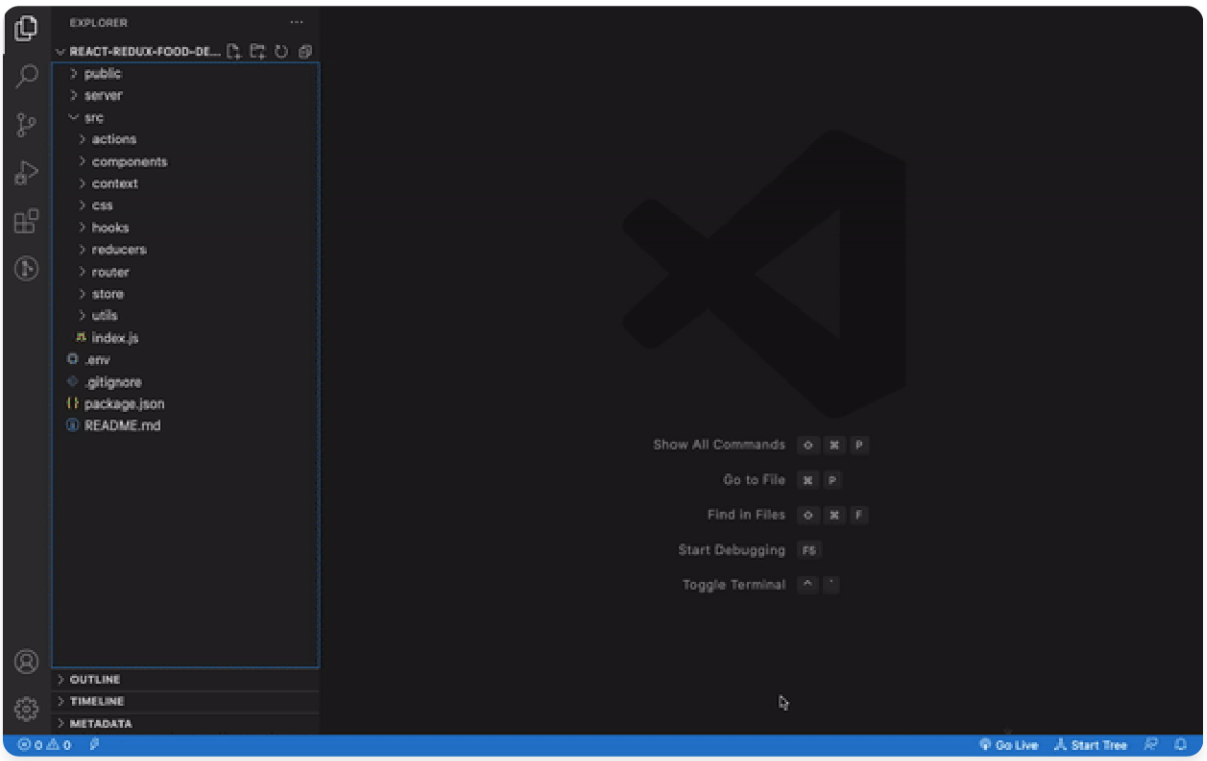
```
// Instead of writing code like this:

const App = () => {
 // ❌ Dont' create queryClient inside component. This is wrong.
 const queryClient = new QueryClient();

 return (
    <QueryClientProvider client={queryClient}>
      <App />
    </QueryClientProvider>
 );
}

// write it like this:

// ✅ This is correct place to create queryClient
const queryClient = new QueryClient();

const App = () => {
 return (
    <QueryClientProvider client={queryClient}>
      <App />
    </QueryClientProvider>
 );
}
```

> *Check out this course, If you want to learn React Query / Tanstack Query From Scratch.*

38. Use ReacTree VSCode Extension To Find Out Parent Child Relationships

Use [ReacTree VS Code extension] to quickly see a tree view of your React application components.

It's a great way to find out the parent and child components relationships.



---

39. Always Use Libraries Like react-hook-form When Working With Forms

Whenever working with simple as well as complex forms in React, I always prefer the [react-hook-form] library which is the most popular React form-related library.

> *It uses refs instead of state for managing input data which helps to avoid performance issues in the application.*

With this library, your component will not re-render on every character typed which is the case when you manage it yourself with the state.

You also get the added benefit of an easy way of implementing reactive validations for your forms.

It also makes your component code a lot simpler and easier to understand.

It's a very powerful library, so even shadcn/ui library uses it for its Form component.

> *Check out this article, If you want to learn react-hook-form from scratch.*

---

40. Avoid Passing setState Function As A Prop To Child Components

Never pass the setState function directly as a prop to any of the child components like this:

```
const Parent = () => {
 const [state, setState] = useState({
   name: '',
   age: ''
 })

 .
 .
 .

 return (
  <Child setState={setState} />
 )
}
```

> *The state of a component should only be changed by that component itself.*

Here's Why:

- This ensures your code is predictable. If you pass setState directly to multiple components, it will be difficult to identify from where the state is getting changed.
- This lack of predictability can lead to unexpected behavior and make debugging code difficult.
- Over time, as your application grows, you may need to refactor or change how the state is managed in the parent component.
- If child components rely on direct access to setState, these changes can ripple through the codebase and require updates in multiple places, increasing the risk of introducing bugs.
- If sensitive data is part of the state, directly passing setState could potentially expose that data to child components, increasing security risks.
- React's component reconciliation algorithm works more efficiently when state and props updates are clearly defined within components.

Instead of passing `setState` directly, you can do the following:

Pass data as props: Pass the data that the child component needs as props, not the setState function itself. This way, you provide a clear interface for the child component to receive data without exposing the implementation details of the state.

Pass function as prop: If the child component needs to interact with the parent component's state, you can pass a function as a prop. Declare a function in the parent component and update the state in that function, you can pass this function as a prop to the child component and call it from the child component when needed.

---

41. Avoid Using Nested Ternary Operators

When using the ternary operator never go beyond one condition and avoid writing nested conditions.

```
// Good Practice
{isAdmin ? <Dashboard /> : <Profile /> }

// Bad Practice
{isAdmin ? <Dashboard /> : isSuperAdmin ? <Settings /> : <Profile />}
```

Using nested conditions when using a ternary operator makes the code difficult to read and maintain even If you're using some formatter to format the code.

If you have more than one condition to check, you can use if/else or switch case or lookup map but never use a ternary operator like this:

```
if (isAdmin) {
 return <Dashboard />;
} else if (isSuperAdmin) {
 return <Settings />;
} else {
 return <Profile />;
}

// OR

switch (true) {
 case isAdmin:
   return <Dashboard />;
 case isSuperAdmin:
   return <Settings />;
 default:
   return <Profile />;
}

// OR

{isAdmin && <Dashboard />}
{isSuperAdmin && <Settings />}
{!isAdmin && !isSuperAdmin && <Profile />}

// OR use a lookup map like this:

const data = {
 isAdmin: <Dashboard />,
 isSuperAdmin: Settings />,
 Neither: <Profile />
}

// and access the data using data['isAdmin'],
// resulting in the Dashboard page being displayed.
```

This applies not only to React but also to JavaScript and other programming languages.

42. Don't Create Separate File For Every Component

You don't have to create every component in a separate file.

If you have a component that is getting used only in a particular file and that component is just returning minimal JSX code, then you can create that component inside the same file.

This avoids unnecessary creation of extra files and also helps to understand code better when it's in a single file.

```
const ProfileDetailsHeader = () => {
  return (
    <header>
      <h1 className='text-3xl font-bold leading-10'>
        Profile Details
      </h1>
      <p className='mt-2 text-base leading-6 text-neutral-500'>
        Add your details to create a personal touch to your profile.
      </p>
    </header>
  );
};

const Profile = () => {
  return (
   <ProfileDetailsHeader />
   ...
   <ProfileDetailsFooter />
  );
}
```

---

43. Never Store Private Information Directly In The Code

I have seen many people directly writing their private information like Firebase configuration in the code like this:

```
const config = {
  apiKey: 'AIdfSyCrjkjsdscbbW-pfOwebgYCyGvu_2kyFkNu_-jyg',
  projectId: 'seventh-capsule-78932',
   ...
};
```

Never ever do this. It's a major security issue. When you push the file with this configuration to GitHub, when someone clones your repository, they're able to directly access your Firebase data.

They can add, edit, or delete the data from your Firebase.

To avoid this issue, you can create a file with the name `.env` in your project and for each property of the config object, create an environment variable like this:

```
// If using Vite.js

VITE_APP_API_KEY=AIdfSyCrjkjsdscbbW-pfOwebgYCyGvu_2kyFkNu_-jyg

// access it as import.meta.env.VITE_APP_API_KEY

// If using create-react-app

REACT_APP_API_KEY=AIdfSyCrjkjsdscbbW-pfOwebgYCyGvu_2kyFkNu_-jyg

// and access it as process.env.REACT_APP_API_KEY
```

Also, add the `.env` file entry to `.gitignore` file so it will not be pushed to GitHub.

---

44. Always Move Your Component Business Logic Into Custom Hooks

Whenever possible, always try to make maximum use of custom hooks.

Always put all your component's business logic, and API calls inside a custom hook.

This way your component code looks clean and easy to understand, maintain, and test.

```
const ResetPassword = () => {
   const { isPending, sendResetEmail, error } = useResetPassword();

   // some JSX
}
```

---

45. Every Use Of Custom Hook Creates a New Instance

Most beginner React developers make this mistake while understanding the custom hooks.

When using a particular custom hook in multiple components, you might think that each component will refer to the same data returned from that hook like this:

```
const [show, toggle] = useToggle();
```

However, this is not the case.

Each component using that hook will have a separate instance of the hook.

As a result, all the states, event handlers, and other data declared in that custom hook will be different for each component using that hook.

So If you need to use the same data and event handlers for all the components, you need to import the custom hook at only one place in the parent component of all of these components.

Then, you can pass the data returned by the custom hook as a prop or use Context API to access it from specific components.

So never make the mistake of using the same custom hook in different components assuming changing hook data from one component will be automatically updated in another component also.

Here's a CodeSandbox Demo to see it in action.

---

46. Avoid Losing State Properties When Updating Objects with React Hooks

When using class components, If you have a state with multiple properties like this:

```
state = {
  name: '',
  isEmployed: false
};
```

Then to update the state we can specify only the property that we want to update like this:

```
this.setState({
  name: 'David'
});

// OR

this.setState({
  isEmployed: true
});
```

So the other state properties will not be lost when updating any property.

But when working with React hooks, If the state stores an object like this:

```
const [state, setState] = useState({
  name: '',
  isEmployed: false
});
```

Then you manually need to spread out the previous properties when updating a particular state property like this:

```
setState((prevState) => {
  return {
    ...prevState,
    name: 'David'
  };
});

// OR

setState((prevState) => {
  return {
    ...prevState,
    isEmployed: true
  };
});
```

If you don't spread out the previous state values, the other properties will be lost.

Because the state is not automatically merged when using the `useState` hook with object.

---

47. Dynamically Adding Tailwind Classes In React Does Not Work

If you're using Tailwind CSS for styling and you want to dynamically add any class then the following code will not work.

```
<div className={`bg-${isActive ? 'red-200' : 'orange-200'}`}>
  Some content
</div>
```

This is because in your final CSS file, Tailwind CSS includes only the classes present during its initial scan of your file.

So the above code will dynamically add the `bg-red-200` or `bg-orange-200` class to the div but its CSS will not be added so you will not see CSS applied to that div.

So to fix this, you need to define the entire class initially itself like this:

```
<div className={`${isActive ? 'bg-red-200' : 'bg-orange-200'}`}>
  Some content
</div>
```

If you have a lot of classes that need to be conditionally added, then you can define an object with the complete class names like this:

```
const colors = {
  purple: 'bg-purple-300',
  red: 'bg-red-300',
  orange: 'bg-orange-300',
  violet: 'bg-violet-300'
};
```

and use it like this:

```
<div className={colors['red']}>
  Some content
</div>
```

---

48. Always Add Default Or Not Found Page Route When Using React Router

Whenever you're implementing routing in React using the React router, don't forget to include a Route for an invalid path(not found page).

Because if someone navigates to any route that does not exist, then he will see a blank page.

To set up an invalid Route, you can mention the invalid Route component as the last Route in the list of Routes as shown below.

```
<Routes>
  <Route path="/" element={<Home />} />
  .
  .
  .
  <Route path="*" element={<NotFoundPage />} />
</Routes>
```

If you don't want to display the NotFoundPage component for an invalid route, you can set up redirection to automatically redirect to the home page for the invalid route as shown below.

```
<Routes>
  <Route path="/" element={<Home />} />
  .
  .
  .
  <Route path="*" element={<Navigate to="/" />} />
</Routes>
```

---

49. Use Spread Operator To Easily Pass Object Properties To Child Component

If you have a lot of props that need to be forwarded to a component, then instead of passing individual props like this:

```
const Users = ({users}) => {
  return (
    <div>
      {users.map(({ id, name, age, salary, isMarried }) => {
        return (
          <User
            key={id}
            name={name}
            age={age}
            salary={salary}
            isMarried={isMarried}
          />
        )
      })}
    </div>
  )
}
```

You can use the spread operator to easily pass props like this:

```
const Users = ({users}) => {
  return (
    <div>
      {users.map((user) => {
        return (
          <User key={user.id} {...user} />
        )
      })}
    </div>
  )
}
```

However, don't overuse this spread syntax. If the object you're spreading has a lot of properties then passing them manually as shown in the first code is better than spreading out.

50. Moving State Into Custom Hook Does Not Stop The Component From Re-rendering

Whenever you're using any custom hook in any of the components and if the state inside that custom hook changes, then the component using that custom hook gets re-rendered.

Take a look at the below code:

```
export const useFetch = () => {
 const [data, setData] = useState([]);
 const [isLoading, setIsLoading] = useState(false);

 // some code to fetch and update data

 return { data, isLoading };
};

const App = () => {
 const { data, isLoading } = useFetch();

 // return some JSX
};
```

As can be seen in the above code, `useFetch` is a custom hook so whenever the `data` or `isLoading` state changes, the `App` component will get re-rended.

So If the `App` component is rendering some other child components, those direct child, as well as indirect child components, will also get re-rendered when the `data` or `isLoading` state from the `useFetch` custom hook changes.

So just because you have refactored the `App` component to move state inside the custom hook does not mean that the `App` component will not get re-rendered.
